

《软件工程导论》复习

整理人：21 软件工程 陈宇森 董佳琪 杨逸哲

22 软件工程 秦薪淇

(课件整理，仅供参考，主体以毛新军版教材为主，下列页码为教材页码)

一、计算机软件及属性 Computer Software and Properties

1.1 软件示例

1.1.1 软件例子

1.1.2 应用领域 (软件分类 p23)

1.2 软件概念 (p16)

1.2.1 软件定义

1.2.2 软件要素

1.3 软件的属性 (p21 软件的特点部分)

1.3.1 产品特性 (p21)

1.3.2 发展挑战

二、软件工程和软件开发过程 (软件工程 p54 开始, 软件开发过程和方法 p95 开始) Software Engineering and Software Development Process

2.1 软件工程 (p58)

2.2 软件过程建模 (p97)

2.3 活动流程

2.4 应对挑战

2.5 流程改进

三、软件工程概论--软件测试与验证 (p408 开始)

3.1 背景 (p55 软件危机)

3.2 软件测试 (p408)

3.3 软件验证

四、软件项目管理和软件维护与发展

4.1 软件项目管理 (p468 开始)

4.2 软件配置管理 (P498 开始)

4.3 软件过程改进

4.4 软件维护和演化 (P451 开始)

五、尾附第四节 version2

一、 计算机软件及属性

1.1 软件不可或缺

1) 信息系统组成：微芯片，计算机硬件与体系结构，计算机网络与通信。

计算机软件：处理数据，提供功能和服务

1.1.1) 软件的地位：不可依赖

2) 软件无处不在：

从工业、国防到日常学习和娱乐，每个领域和领域都需要软件。在过去的 50 年里，开源软件系统已经积累了 100TB 的代码，而在过去的 5000 年里，人类写的书的数量只有 23TB。

3) 软件是创造性的工具：

TikTok 是一个创造性的软件工具； 华为已经成为一家软件密集型企业

4) 软件的深远影响：

交通 12306, Ctrip; 搜索引擎 谷歌 百度 教育 MOOC 腾讯

购物：淘宝 办公室：Word、PPT 支付手段：微信、支付宝

5) 软件是军事装备和信息系统的核心：



越来越多的软件表现为人-机-物共生的一类复杂系统

6) 机器人是一个软件密集型系统：

示例：机器人是一类由软件驱动的物理系统

□ 机器人是由软件驱动、由软件来定义的软件密集型信息物理系统

✓ 负责底层管理和控制、高层规划和决策、全局适应和优化等

□ 软件是整个系统的核心和关键



7) 软件在武器系统中的地位：

武器系统中 80% 的功能由软件实现； 软件密集型系统的数量不断增长

8) 例：作战指挥和控制中的软件

实现 C4ISR：

指挥 Command、控制 Control、通信 communication、计算 computing、情报 information、监视 surveillance、侦查 reconnaissance

- ✓ 达成战场信息共享和展示
- ✓ 支持和辅助作战决策
- ✓ 促进不同系统的集成
- ✓ 实现体系化，互联互通互操作软件规模的增加：

1.1.2) 软件的规模和复杂性日趋增长

1) 软件变得越来越复杂

□ 软件规模带来的复杂性

- ✓ 大量的软件实体、数据、交互、进程等带来的复杂性

□ 软件环境的复杂性

- ✓ 软件表现为人机物三元融合复杂系统
- ✓ 软件需要与物理环境、社会环境等进行持续交互
- ✓ 软件所在的环境开放、动态、不确定和不可预测

□ 软件需求的复杂性

- ✓ 软件需求不可知、不确定和动态变化

□ 软件运维的复杂性

- ✓ 软件需要在使用的同时持续演化

2) 大规模和复杂软件带来的挑战？

- π 软件复杂性表现在哪些方面？
- π 如何来开发复杂的软件系统？
- π 需要哪些方面的知识和技能？

美国军方对软件的认识

美国军方向软件工程学术和产业界提出的问题

面临具有十亿行代码规模的软件，这样的系统今天的技术力所不能及



The Honorable
Clayton K. Green, Jr.

我们的士兵依赖软件并且将越来越多的依赖软件
军队的成功依赖于软件以及软件企业
我们需要更好的工具来满足未来的挑战，我们的政府和企业并没有致力于如何让事情做的快速和廉价
如果我们不能解决今天的问题，我们如何能可靠地构建未来可能具有十亿行代码的系统

3) 失败的软件项目和产品：

为什么失败？未能按时交付；成本超出预算；质量差

- 在软件开发中有哪些困难？
- 如何开发一个软件系统？
- 为什么开发成本如此高？
- 为什么会有这么多失败的项目？

1.2) 软件的概念：

软件是由程序、数据和文档组成的集合，借助计算机系统可以提供特定功能的工具。

1.2.1) 什么是文档

设计文档是记录开发活动和结果、配置和变更的描述性材料。包含软件需求；软件设计；软件测试

1.2.2) 写设计文档的原因和目的

原因：

软件开发涉及多个活动；保存开发活动和结果的记录；分析开发结果

目的:

清晰的描述; 发现问题; 开发者之间的沟通; 开发成果管理
什么是数据? 数据是程序处理的对象和结果

1.2.3) 什么是数据:

数据是程序处理的对象和结果。

数据的形式: 用户、交易、日志等。

数据处理: 表示, 获取, 存储, 检索, 分析

1.2.3.0) 从开发者的角度看软件定义: 软件 software 不等于程序 program ; 软件开发 software development 不等于编程 coding

从开发者的角度来看, 软件的定义是一个更广泛的概念, 它不仅包括程序 (即编写的代码), 还包括数据和文档等内容, 以及与计算机系统相关的各种元素。从这个角度来看, 软件开发确实不等于编程, 因为软件开发涉及到整个软件生命周期的方方面面, 包括需求分析、设计、测试、部署和维护等阶段, 而不仅仅局限于编写程序代码这一部分。因此, 软件开发是一个更全面的过程, 而编程只是其中的一部分内容。

1.2.3.1) 软件开发的反思:

开发大型复杂的软件与编写简单的程序是不同的; 寻求有效的途径——工程方法;

1.2.4) 软件生命周期 (p19):

从软件开始开发到结束的一段时间;

包含多个阶段: 需求分析-》软件设计-》编码-》测试-》维护。

每个阶段都有自己的任务、结果和工作

1.2.5) 软件的属性:

逻辑的产品; 复杂性; 设计开发; 故障难可见; 可变性

1.2.6) 软件类别:

应用软件; 系统软件; 支持软件

1.2.6.1) 闭源软件 CSS:

软件的源代码不向用户开放。要提供使用该软件的许可;

潜在问题: 不知道恶意代码; 无法更改代码 比如: windows ; office

开源软件 OSS:

软件的源代码可以自由获取和传播。软件代码的所有者在许可的范围内允许使用、修改和传播软件的源代码

优点: 自由传播; 激发热情; 降低开发成本 比如: Linux, Ubuntu, Eclipse, Apache, MySQL

1.2.6.2) OSS 取代 CSS

服务器操作系统 Linux、FreeBSD vs Unix;

桌面操作系统 Linux vs Windows;

数据库 MySQL vs Oracle;

浏览器 Chrome, Firefox vs IE;

开发工具 Eclipse vs Visual Studio

Discussion:

❑ What are benefit of OSS?

❑ How to develop OSS?

❑ Why so many enterprises and developers follow with interests and participate in OSS?

1.2.6.3) 我们软件工业的挑战

在许多软件领域中感到薄弱是一个常见的问题, 特别是在以下几个方面:

- ✓系统软件，例如操作系统（OS）
- ✓支持软件，例如 CASE 工具
- ✓行业软件，例如 EDA 软件

解决方案：

- ✓长期投资和参与
- ✓培养优秀的软件工程师

软件质量分为 修改；传输；通过 这三个方面

可维护性可理解性；可移植性、可重用性、互操作性；正确性，效率

我国软件业的挑战：

在许多软件领域很弱：系统软件，如操作系统;支持软件，如 CASE 工具;工业软件，如 EDA 软件

解决方案：长期投资和参与培养优秀的软件工程师

大规模和复杂软件带来的挑战？

Q1 软件复杂性表现在哪些方面？

1. 功能复杂性：软件系统通常需要实现多个功能，这些功能之间的相互关系和交互可能非常复杂。
2. 数据复杂性：软件系统需要处理大量的数据，包括输入、输出以及中间数据。这些数据可能存在不同的格式、类型和结构，处理这些数据的过程可能会很复杂。
3. 逻辑复杂性：软件系统需要实现一定的逻辑，包括流程控制、条件判断、循环等。随着软件规模的增大，逻辑复杂性也会增加。
4. 接口复杂性：软件系统可能需要与其他系统或者硬件设备进行交互，这涉及到接口的设计和实现。接口的复杂性可能源于不同系统之间的协议、数据格式等差异。
5. 可变性复杂性：软件系统通常需要满足不同用户的需求，因此需要具备一定的灵活性和可扩展性。这种可变性可能导致软件系统的复杂性增加。

Q2 如何来开发复杂的软件系统？

1. 需求分析：充分理解用户需求，明确软件系统的功能和性能要求。
2. 架构设计：设计合理的软件架构，将系统划分为多个模块或组件，明确各个模块之间的关系和接口。
3. 模块化开发：采用模块化的开发方式，将复杂的问题拆分为多个简单的子问题，逐个解决。
4. 测试与验证：进行全面的测试和验证，确保软件系统的功能和性能符合预期。
5. 文档管理：及时记录软件系统的设计、实现和使用说明，方便后续维护和升级

Q3:需要哪些方面的知识和技能？

1. 编程语言和工具：熟悉至少一种编程语言，并掌握相关的开发工具和集成开发环境（IDE）。
2. 数据结构与算法：对数据结构和算法有深入的理解，能够选择和应用适当的数据结构和算法来解决问题。
3. 软件工程知识：了解软件开发的基本原理和流程，包括需求分析、设计、开发、测试和维护等阶段。
4. 架构设计和设计模式：了解常用的软件架构设计方法和设计模式，能够选择合适的架构和模式来组织代码。

5. 沟通协作能力：良好的沟通和协作能力，能够与团队成员、用户和其他利益相关者有效地进行交流与合作。
6. 故障排除与调试：具备故障排除和调试的技能，能够快速定位和解决软件系统中的问题。
7. 持续学习和更新：软件行业发展迅速，需要保持学习的状态，不断更新自己的知识和技能。

Q4 在软件开发中有哪些困难？

1. 复杂性：软件系统通常是复杂的，涉及多个模块、功能和交互。处理这些复杂性可能导致设计、开发和测试上的困难。
2. 变更管理：软件需求和规格经常发生变化，需要及时响应和适应变化。但是，变更管理可能增加开发的复杂性和成本。
3. 资源限制：软件开发需要人力、时间和金钱等资源投入。资源限制可能导致项目进度延误、质量下降或功能不完整。
4. 技术挑战：某些软件开发任务可能涉及新技术、算法或领域知识。对于开发团队来说，理解和应用这些新技术可能是一项挑战。
5. 沟通与协作：软件开发通常是一个团队合作的过程，需要良好的沟通和协作。缺乏有效的沟通和协作可能导致项目延误或产生误解。
6. 质量保证：确保软件系统的质量和稳定性是一个挑战。测试、调试和验证的工作可能很复杂，需要充分的时间和资源。
7. 用户满意度：软件开发的最终目标是满足用户需求和期望。但是，理解和满足用户需求可能是困难的，需要进行充分的需求分析和用户反馈。

Q5: □ 如何开发一个软件系统？

Q6: □ 为什么开发成本如此高？

1. 人力资源成本：软件开发需要雇佣有技术能力的开发人员，他们的工资和福利支出较高。
2. 开发工具和设备成本：软件开发需要使用各种开发工具和设备，包括开发软件、硬件设备和测试工具等。
3. 项目管理成本：对项目进行规划、管理和监控需要一些成本投入，包括项目经理和项目管理工具等。
4. 质量保证成本：为了确保软件系统的质量和稳定性，需要进行各种测试、验证和调试工作，这些工作也需要一定的成本支出。
5. 变更管理成本：软件需求和规格变更可能导致额外的开发和测试工作，增加了开发成本。

Q7 □ 为什么会有这么多失败的项目？

1. 需求不清晰：需求分析不充分或不准确，导致开发过程中出现功能缺陷或用户不满意。
2. 范围控制不当：软件项目的范围无法有效控制，导致进度延误和超出预算。
3. 沟通问题：开发团队内部或与用户之间的沟通不畅，导致误解和冲突。
4. 技术挑战：遇到了难以解决的技术问题，导致项目无法按计划完成。
5. 缺乏风险管理：未能及时发现和应对项目中的潜在风险，导致项目失败。
6. 管理问题：项目管理不善，导致资源分配不合理、进度控制不当等问题。
7. 缺乏培训和支持：对于新技术或新工具的使用，团队成员可能缺乏培训和支持，导致项目失败。

Q8: ☐What are benefit of OSS?

1. 降低成本:使用开源软件可以节省企业的软件开发和购买成本,降低软件采购成本。
2. 灵活性:开源软件通常具有可定制和可扩展性,可以根据企业需求进行自定义开发。
3. 透明度:开源软件的代码是公开的,用户可以查看和修改代码,从而更好地了解软件的功能和运行原理。
4. 社区支持:开源软件通常有庞大的社区支持,用户可以从社区获取技术支持、知识分享和问题解决方案等。
5. 安全性:由于开源软件的代码是公开的,任何人都可以对其进行安全审计和漏洞修复,从而提高软件的安全性。

Q9 ☐How to develop OSS?

1. 选择合适的许可证:开源软件需要选择合适的开源许可证,以确保代码的开放性和合法性。
2. 设计软件架构:开源软件需要有清晰的软件架构,以便其他开发者参与进来进行维护和开发。
3. 编写代码:开源软件的代码需要遵循编码规范和最佳实践,确保代码质量和可读性。
4. 提供文档和测试:开源软件需要提供充分的文档和测试,方便用户使用和开发。
5. 建立社区:开源软件需要建立活跃的社区,与其他开发者共同维护软件,并提供技术支持和知识分享。

Q10☐Why so many enterprises and developers follow with interests and participate in OSS?(为什么很多企业和开发者对开源软件感兴趣并参与其中:)

1. 开源软件可以降低企业的开发和采购成本,提高效率。
2. 开源软件具有灵活性和可定制性,可以根据企业需求进行自定义开发。
3. 开源软件的代码是公开的,可以避免厂商锁定和依赖。
4. 开源软件有庞大的社区支持,可以获取技术支持、知识分享和问题解决方案等。
5. 参与开源软件开发可以提高开发者的技术能力和知名度。

二、 软件工程与软件开发过程

2.1 软件工程

2.1.1) 软件危机: 软件开发和维护过程中出现了许多问题, 软件质量, 开发成本, 推迟截止日期, 难以维护

2.1.2) 软件成本方面: 个人电脑上的软件成本通常高于硬件成本。软件成本往往占计算机系统成本的大头。软件的维护成本高于开发成本。对于寿命较长的系统, 维护成本可能是开发成本的几倍。软件工程关注的是具有成本效益的软件开发。

2.1.3) 软件项目的失败:

增加的系统复杂性:

系统必须更快地建立和交付,需要更大、更复杂的系统,系统必须具有以前认为不可能的新功能。

未能使用软件工程方法:

不使用软件工程方法和技术编写计算机程序是相当容易的,许多公司在日常工作中并不使用软件工程方法。

2.1.4) 软件工程常见问题:

1 什么是软件?

计算机程序和相关文件。软件产品可以为特定客户开发, 也可以为一般市场开发。

2 好的软件有哪些属性?

好的软件应该向用户交付所需的功能和性能, 并且应该是可维护的、可靠的和可用的。

3 什么是软件工程?

软件工程是一门涉及软件生产各个方面的工程学科。

4 什么是基本的软件工程活动?

软件规范、软件开发、软件工程和软件的演进。

5 软件工程和计算机科学之间的区别是什么?

计算机科学侧重于理论和基础;软件工程关注开发和交付有用软件的实用性。

6 软件过程与系统工程有什么区别?

系统工程涉及基于计算机的系统开发的各个方面, 包括硬件、软件和过程工程。软件工程是这个更普遍的过程的一部分。

7 软件工程面临的主要挑战是什么?

应对日益增加的多样性, 需求减少交付时间和开发值得信赖的软件。

8 软件工程的成本是什么?

大约 60%的软件成本是开发成本, 40%是检测费用。对于定制软件, 进化成本通常超过开发成本。

9 最好的软件工程技术和方法是什么?

不同的技术适用于不同类型的系统。例如, 游戏应该始终使用一系列原型来开发, 而安全关键控制系统则需要开发一个完整且可分析的规范。

10 网络给软件工程带来了什么不同?

网络导致了软件服务的可用性和开发高度分布式的基于服务的系统的可能性。基于 web 的系统开发在编程语言和软件重用方面取得了重要进展。

2.1.5)软件产品:

通用产品:

销售给任何想要购买它们的客户的独立系统。

示例- PC 软件如图形程序、项目管理工具;CAD 软件;针对特定市场的软件, 如牙医预约系统。
定制化产品:

由特定客户委托以满足其自身需求的软件。

例子-嵌入式控制系统, 空中交通管制软件, 交通监控系统。

2.1.6)优秀软件的基本属性:

- 1 可维护性: 软件应该以这样一种方式编写, 以便它能够发展以满足客户不断变化的需求。这是一个关键属性, 因为软件更改是不断变化的业务环境的不可避免的需求。
- 2 可靠性和安全性: 软件可靠性包括一系列特征, 包括可靠性、安全性和安全性。在系统发生故障时, 可靠的软件不应造成物理或经济损失。恶意用户不应该能够访问或破坏系统。
- 3 效率: 软件不应该浪费系统资源, 如内存和处理器周期。因此, 效率包括响应性、处理时间、内存利用率等。
- 4 可接受性: 软件必须为其设计的用户类型所接受。这意味着它必须是可理解的、可用的, 并且与他们使用的其他系统兼容。

2.1.7)软件工程:

软件工程是一门工程学科, 它涉及软件生产的各个方面, 从系统规格说明的早期阶段到系统投入使用后的维护。

工程学科:

运用适当的理论和方法解决组织和财务约束下的问题。

软件生产的各个方面

不仅仅是技术的发展过程。项目管理和开发工具, 方法等, 以支持软件生产。

2.1.8) SE 重要性:

个人和社会越来越依赖于先进的软件系统。我们需要能够经济而快速地生产可靠和值得信赖的系统。从长远来看, 在软件系统中使用软件工程方法和技术, 而不是像编写个人编程项目那样编写程序, 通常更便宜。对于大多数类型的系统, 大部分成本是在软件投入使用后更改软件的成本。

2.1.9)软件工程多样性: 有许多不同类型的软件系统, 没有一套通用的软件技术适用于所有这些系统。客户需求和开发团队背景。所使用的软件工程方法和工具取决于所开发的应用程序的类型

2.1.10)软件工程基础知识:

一些基本原则适用于所有类型的软件系统, 而不考虑所使用的开发技术:

应该使用可管理和可理解的开发过程来开发系统。不同类型的软件使用不同的进程。

可靠性和性能对所有类型的系统都很重要。

理解和管理软件规范和需求(软件应该做什么)是很重要的。

在适当的情况下, 应该重用已经开发的软件, 而不是编写新的软件。

2.2)软件过程模型 (p95 开始)

2.2.1)软件流程:

开发软件系统所需的一组结构化活动。

许多不同的软件过程, 但都涉及:

规范——定义系统应该做什么;

设计与实施——定义系统的组织结构并实施系统;

验证——检查产品是否符合客户的要求;

进化——改变系统以应对变化。

软件过程模型是过程的抽象表示。

2.2.2) 软件流程描述:

当我们描述和讨论流程时, 我们通常谈论这些流程中的活动, 如指定数据模型、设计用户界面等, 以及这些活动的顺序。

过程描述还可以包括:

产品, 即过程活动的结果;

角色, 反映过程中涉及的人员的责任;

前置和后置条件, 它们是在流程活动实施或产品生产之前和之后成立的语句。

2.2.3) 计划驱动和敏捷过程:

计划驱动的过程是所有过程活动都预先计划好的过程, 并且根据该计划度量进展。

在敏捷过程中, 计划是增量的, 并且更容易更改过程以反映不断变化的客户需求。

在实践中, 大多数实际过程包括以下元素: 计划驱动和敏捷方法。

没有正确或错误的软件过程。

2.2.4) 软件过程模型:

瀑布模型: 计划驱动的模式。规范和开发的分离和不同阶段。

增量开发: 规范、开发和验证是相互交织的。可能是计划驱动的, 也可能是敏捷的。

集成和配置: 该系统由现有组件组装而成。可能是计划驱动的, 也可能是敏捷的。

可配置的在实践中, 大多数大型系统都是使用一个包含所有这些模型元素的过程来开发的。

2.2.4.1) 瀑布模型问题:

将项目不灵活地划分为不同的阶段使得很难对不断变化的客户需求做出反应因此,

只有当需求被充分理解, 并且在设计过程中更改是相当有限的时候, 这个模型才是合适的。

很少有业务系统有稳定的需求。

瀑布模型主要用于在多个地点开发系统的大型系统工程项目。

在这些情况下, 瀑布模型的计划驱动性质有助于协调工作。

2.2.4.2) 增量开发

优点:

适应不断变化的客户需求的成本降低了。

对于已经完成的开发工作, 更容易获得客户反馈。

向客户更快地交付和部署有用的软件是可能的。

问题:

进程是不可见的。管理者需要定期交付成果来衡量进度。如果系统是快速开发的, 那么生成反映系统每个版本的文档是不划算的。

随着新增量的增加, 系统结构趋于退化。除非花费时间和金钱进行重构以改进软件, 否则定期的更改往往会破坏其结构。合并进一步的软件更改变得越来越困难和昂贵。

2.2.4.3) 集成和配置:

基于软件重用, 其中系统从现有组件或应用程序系统(有时称为 COTS - 商用现货)系统集成。

可以对重用的元素进行配置, 使其行为和功能适应用户的需求;

重用现在是构建多种类型业务系统的标准方法

可重用软件的类型: 为在特定环境中使用而配置的独立应用程序系统(有时称为 COTS)。作为包开发的对象集合, 以便与 .net 或 J2EE 等组件框架集成。根据服务标准开发的 Web 服务, 可用于远程调用。

面向重用的软件工程: 降低成本和风险, 因为从零开始开发的软件更少更快的交付和部署系统但是需求妥协是不可避免的, 因此系统可能无法满足用户的真正需求失去对可重用系统元素演变的控制

2.3) 流程活动

真正的软件过程是技术、协作和管理活动的交错序列，其总体目标是指定、设计、实现和测试软件系统。

在不同的开发过程中，规范、开发、验证和演进这四个基本过程活动的组织方式是不同的。

例如，在瀑布模型中，它们是按顺序组织的，而在增量开发中，它们是交错的。

2.3.1)软件规范：确定需要哪些服务以及系统运行和开发的约束条件的过程。

需求工程过程：

需求引出和分析系统：股东对系统有什么要求或期望？

需求规范：详细定义需求

需求验证：检查需求的有效性

2.3.2) 软件设计与实现：

将系统规范转换为可执行系统的过程。

软件设计

设计一个实现规范的软件结构；

实现

将此结构转换为可执行程序；

设计和实施的活动是密切相关的，可能是相互交织的。

1)设计活动：

体系结构设计，在此您确定系统的总体结构、主要组件(子系统或模块)、它们的关系以及它们是如何分布的。

数据库设计，设计系统数据结构以及如何在数据库中表示这些结构。

接口设计，定义系统组件之间的接口。

组件选择和设计，其中搜索可重用组件。如果不可用，则设计它的操作方式。

2)系统实施：

该软件可以通过开发一个或多个程序或配置应用系统来实现。

编程是一种独立的活动，没有标准的过程。

调试是发现程序错误并纠正这些错误的活动。

2.3.3)软件验证：

验证和确认旨在表明系统符合其规范并满足系统客户的要求。测试。

包括检查和评审过程和系统系统测试

包括使用测试用例来执行系统，这些测试用例来源于系统要处理的真实数据的规范。

测试是最常用的（V & V）活动。

1)测试阶段：

组件测试

单个组件独立测试;组件可以是功能或对象，也可以是这些实体的一致分组。

系统测试

对整个系统进行测试。测试涌现特性尤为重要。

客户测试

使用客户数据进行测试，检查系统是否满足要求。

2.3.4)软件演化：

软件本质上是灵活的，可以改变的。

随着需求随着业务环境的变化而变化，支持业务的软件也必须发展和变化。

尽管在开发和进化(维护)之间已经有了界限，但随着越来越少的系统是全新的，这一点越来越无关紧要。

2.4)应对变化

在所有大型软件项目中，变化是不可避免的。

业务变更导致新的和变更的系统需求

新技术为改进实现提供了新的可能性

不断变化的平台需要应用程序的变化

变更导致返工，因此变更的成本既包括返工(例如，重新分析需求)，也包括实现新功能的成本。

1)应对不断变化的需求：

系统原型，快速开发系统的一个版本或系统的一部分，以检查客户的需求和设计决策的可行性。这种方法支持变更预期。

增量交付，将系统增量交付给客户以供评论和实验。这既支持更改避免，也支持更改容忍。

2.4.1)软件原型：

原型是系统的初始版本，用于演示概念和尝试设计选项。

原型可用于：

需求工程过程，以帮助需求的引出和验证；

在设计过程中探索选项并开发 UI 设计；

在测试过程中运行背靠背测试。

1)原型的好处：提高系统可用性。更贴近用户的真实需求。提高设计质量。提高可维护性。减少开发工作。

2)原型开发：

可能基于快速原型语言或工具；

可能会遗漏功能原型：

应该关注产品中尚未被很好理解的部分；

错误检查和恢复可能不包括在原型中；

关注功能性需求而不是非功能性需求，比如可靠性和安全性。

2.4.2)增量交付：

不是将系统作为单个交付交付，而是将开发和交付分解为增量，每个增量交付所需功能的一部分。

用户需求被划分优先级，最高优先级的需求被包含在早期的增量中。

一旦增量的开发开始，需求就被冻结了，尽管后续增量的需求可以继续发展。

1) 增量开发和交付：

开发：

以增量的方式开发系统，并在进行下一个增量的开发之前评估每个增量；

敏捷方法中常用的常规方法；

由用户/客户代理完成的评估。

交付：

部署一个增量供最终用户使用；

对软件的实际使用进行更现实的评估；

对于替代系统来说，实施起来较为困难，因为增量的功能比被替代的系统少。

2) 增量交付优点：

客户价值可以随每个增量一起交付，这样系统功能就可以更早地使用。

早期的增量作为原型来帮助引出后期增量的需求。

降低整个项目失败的风险。

优先级最高的系统服务往往会接受最多的测试。

2.5) 过程改进

许多软件公司已经将软件过程改进作为提高软件质量、降低成本或加速开发过程的一种方法。

过程改进意味着理解现有的过程并改变这些过程以提高产品质量和/或减少成本和开发时间。

1) 改进方法：

过程成熟度方法，侧重于改进过程和项目管理，并引入良好的软件工程实践。

过程成熟度的级别反映了在组织软件开发过程中采用良好技术和管理实践的程度。

敏捷方法，侧重于迭代开发和减少软件过程中的开销。客户的需求。

敏捷方法的主要特征是功能的快速交付和对变化的响应。

2) 过程改进活动：

过程测量：

您度量软件过程或产品的一个或多个属性。这些度量形成了一个基线，帮助您确定过程改进是否有效。

过程分析：

评估当前过程，确定过程的弱点和瓶颈。可以开发描述过程的过程模型(有时称为过程映射)。

过程变更：

提出流程变更以解决一些已确定的流程弱点。引入这些内容后，循环继续收集有关更改有效性的数据。

3)过程度量：

完成流程活动所花费的时间

eg 完成一项活动或过程的时间或努力。

过程或活动所需的资源：

eg 以人-天为单位的总工作量。

特定事件发生的次数：

eg 发现的缺陷数量。

三、软件工程概论--软件测试与验证

3.1) 背景:

3.1.1) 软件危机

低质量和高成本; 增加的复杂性; 多开发者合作

3.1.2) 软件质量保证:

开发过程

软件测试: bug 检测 (软件测试是工业中使用最广泛的方法) #

软件验证: 无 bug 存在

3.2) 软件测试

1) 与特定操作环境的单点相等可能导致分支的错误执行。

2) 变量类型的不当使用可能导致死循环。

3.2.1) 基础概念:

软件故障 fault: 静态 Static 缺陷软件(缺陷)

软件错误 Error: 不正确的内部状态, 执行缺陷引起。

软件失败 Failure: 结果与预期的外部观察不匹配

进一步: 错误是由人为因素引起的, 故障是实际代码中存在的问题, 而失效是用户在软件使用过程中遇到的问题

Fault (故障): 是指在软件中存在的缺陷或错误。故障是指实际代码中存在的问题, 是由于错误引起的。故障可以由不正确的设计、编码错误或其他技术因素引起的。

Error (错误): 是指在编写或设计软件时的人为错误或误差。这是人类行为导致的问题。例如, 程序员可能会犯语法错误、逻辑错误或算法错误。

Failure (失效): 是指软件在运行时无法按预期工作或提供正确的输出。失效是指软件系统在特定条件下产生了错误结果或不符合预期的行为。失效是用户可见的, 可能导致系统崩溃、功能无法正常运行或输出不正确。

3.2.2) 失效模型:

执行/可达性: 必须到达程序中包含缺陷的一个或多个位置。

感染: 程序的状态必须不正确

传播: 受错误影响的状态 必须传播, 导致程序的某些输出不正确

2) 测试 test vs 调试 debug

测试: 测试通过执行和观察失败来揭示缺陷

静态测试: 不执行程序

动态测试: 执行程序

调试: 调试是通过定位、理解和纠正缺陷来修复缺陷的过程

3.3) 软件验证

1) 定义: 证明程序满足某些性质的活动

通常使用数学推理(或形式推理)

手动 vs 自动

- 2)历史：几乎与计算机科学同时发展起来
- 3)常用软件验证方法：定理证明；模型检查
- 4)定理证明：演绎推理，如霍尔逻辑

基于 SMT 的自动方法；
人机交互的半自动方法；
手动方法

5)定理证明工具

半自动:Dafny, Why3, Verifast, Smallfoot
手动:Coq, Isabelle/HOL

6)模型检查：模型检验验证了两者之间的满足关系一个程序及其属性规范由自动执行遍历程序的有限状态空间

7)模型检查工具

型号级别:SPIN, SMV, UPPAAL

代码等级:ESC/Java, SLAM, BLAST, CPA Checker, Seahorn

Q1:为什么软件会导致如此多的灾难后果呢？

1. 软件错误：软件中的代码错误、逻辑错误或设计缺陷可能导致系统崩溃、数据丢失或不正确的操作。这些错误可能是开发人员在编写代码时的失误，也可能是由于需求理解错误或系统规模复杂性导致的。
2. 安全漏洞：软件中存在的安全漏洞可能被黑客或恶意用户利用，从而导致数据泄露、身份盗窃、网络攻击等安全问题。这些漏洞可能是由于开发过程中的错误实现、不完善的授权验证或不安全的配置造成的。
3. 不可预见的环境变化：软件往往在复杂多变的环境中运行，例如不同的操作系统、硬件设备或依赖的外部服务。当这些环境发生变化时，软件的行为可能变得不可预测，导致灾难性的后果。
4. 人为错误：人为错误，如配置错误、误操作、恶意行为或管理失误等，也可能导致软件造成严重后果。这些错误可能是由于开发人员、运维人员或用户的疏忽或不当行为引起的。

Q2: 为什么无法在软件交付使用之前消除所有错误呢？

1. 复杂性：现代软件往往非常复杂，涉及大量的代码、模块和交互，同时还要考虑不同的操作系统、硬件和网络环境。这种复杂性使得很难在开发过程中捕捉到所有潜在的错误和边界情况。
2. 时间和成本限制：软件开发通常有时间和成本的限制。为了满足市场需求或项目计划，开发团队可能需要在紧凑的时间框架内交付软件。这可能导致一些测试和错误修复的工作被简化或推迟，从而增加了交付时存在错误的风险。
3. 不完善的需求理解：软件开发通常开始于对需求的理解和规划。然而，需求可能会发生变化、不完整或不明确，这可能导致开发人员无法准确地捕捉到所有的功能和用户需求，从而导致错误的出现。
4. 人为因素：软件开发涉及多个人员的协作，包括开发人员、测试人员和项目管理人员等。由于人为因素，如疏忽、误解或沟通问题，可能导致一些错误未被及时发现或修复。

Q3: 在软件交付和使用之前，有哪些方法可以用来尽可能地检测出更多的错误？

1. 自动化测试：采用自动化测试工具进行单元测试、集成测试和系统测试，以确保软

件的各个部分都能按预期工作。自动化测试可以帮助发现代码中的逻辑错误、边界情况和异常情况。

2. 手动测试：通过手动测试来模拟真实的用户操作和使用场景，发现软件中的交互问题、用户体验问题和功能缺陷。手动测试通常涉及功能测试、用户界面测试和性能测试等。
3. 静态代码分析：使用静态代码分析工具来检测代码中的潜在问题，如未使用的变量、未处理的异常、代码重复等。这有助于提前发现一些常见的编程错误和不良实践。
4. 安全审计：进行安全审计和漏洞扫描，以发现软件中存在的安全漏洞和潜在的攻击面。通过对软件进行渗透测试和安全漏洞扫描，可以提前发现并修复安全风险。
5. 用户反馈和 Beta 测试：积极收集用户反馈，并通过 Beta 测试邀请用户参与软件的试用和反馈。用户反馈可以帮助发现一些特定环境下的问题和用户关注的痛点。
6. 持续集成和持续交付：采用持续集成和持续交付的方式，将代码频繁地集成到主干分支，并进行自动化构建、测试和部署。这有助于及早发现新代码引入的问题。

Q4: the worst software you have used?

1. 非直观的用户界面：设计不良或令人困惑的用户界面的软件可能使使用者感到沮丧，并阻碍生产力。
2. 错误和稳定性问题：经常崩溃、冻结或表现出意外行为的软件会令人沮丧和不可靠。
3. 缺乏功能或功能：缺少基本功能或未能满足用户期望的软件可能无法充分实现其预期目的。
4. 性能不佳：慢或效率低下的软件可能令人沮丧，特别是当它显著影响生产力或响应能力时。
5. 支持和更新不足：缺乏及时更新、补丁或适当的技术支持的软件可能会使用户陷入无法解决的问题或漏洞中。

Q5:The most difficult bug that you have debugged?

1. Heisenbugs：这些 bug 在尝试进行调试时似乎会改变或消失，使其特别具有挑战性，难以识别和修复。
2. 竞态条件：这些 bug 发生在多个线程或进程以意外的顺序访问共享资源时，导致不可预测的行为，很难复制和诊断。
3. 内存相关的 bug：内存泄漏、空指针引用、缓冲区溢出或其他内存相关问题可能很难追踪和修复，因为它们通常会导致间歇性崩溃或不可预测的程序行为。
4. 复杂的交互：由系统的不同组件或模块之间的复杂交互产生的 bug 可能需要大量的工作来识别和解决。
5. 平台或环境特定的 bug：只在某些平台、操作系统或硬件配置上出现的 bug 如果开发环境与生产环境不同，则可能难以重现和调试。

四、 软件项目管理和软件维护与演化

4.1 软件项目管理

1.项目是一种创造特定产品或服务的努力。项目是基于给定的资源和约束条件，为实现一组目标而进行的活动。(课本 p468 项目的概念) (比如三峡工程，探月工程，华为鸿蒙系统)

(项目，又称工程，有约束，有实施活动，提供服务和产品)

2.项目的特点：目标性，进度性，约束性，多方性，独立性和不确定性 (课本 p469)

3.影响项目成功的四方面因素：范围，时间，成本，质量 (课本 p469)

4.何为软件项目？

“软件开发活动”旨在创建一种名为“软件”的特定产品或服务。在开发对象，过程，属性，复杂性等方面，具有 (1) .开发对象:一个逻辑产品“软件” (2) .过程:不注重制造，不存在重复生产流程 (3) .属性:实施要素，如成本，进度，质量，难以衡量， (4) .复杂性:作为逻辑产品的高复杂性 (5.易变性:软件需求很难确定，经常会发生变化 (课本 p470)

5.软件项目的任务 (p469)

6.软件项目举例 (微信，12306，卫星图像加工软件，潜艇控制软件…)

7.军用软件特点：

1) .形式上通常与特定的物理或硬件环境、人机和物体融合系统有关

2)在要求上由于环境的恶劣性和多变性以及军用软件对质量的更高要求，对可靠性、安全性和灵活性提出了更高的要求

由于需要及时响应和应对事件，军用软件项目对系统的实时性和软件的复杂性提出了更高的要求

3) 在复杂程度上随着信息技术的发展，军用软件项目的规模越来越大，甚至可能是另一个系统，或者是超大规模系统

8.软件项目所涉及对象 (p470)。

概念：对软件项目中的人员、产品、过程进行度量、分析、计划、组织和控制，以确保软件项目按照预定的成本、进度和质量要求顺利完成的过程 (p470)

主要内容：(p471 表格中对应管理要素)

4.2 软件配置管理 (P498 开始)

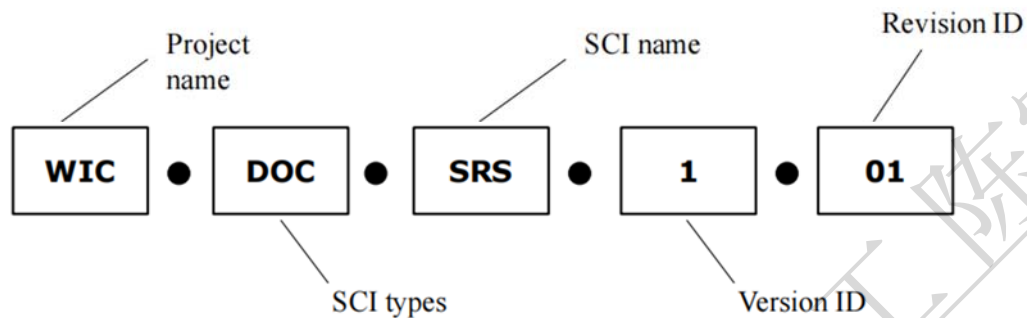
1.配置项是软件配置项(SCI)是在软件生命周期中产生的需要配置和管理的工作制品，包括技术文档，管理文档，程序代码，数据，标准和规约 (具体解释见 p498 下)

2.软件配置管理是指在软件生存周期中对软件制品采取的以下一系列活动的过程：1) 系统控制配置项的标识、存储、修改和发放；2) 记录并报告其状态 3) 验证配置项的正确性和一致性 4) 审记上述工作 (p498 中部)

3.基线是一种已经正式审查和批准的软件产品、标准或规范，它们可以作为进一步开发的基础。变更基线只允许通过正式的变更控制过程。(p499)

□ Generate a unique and intuitive identity for each configuration item

WIC.DOC.SRS.1.01



4.配置项标识：(p501) 上图在 501 页有完整翻译和示例。

5.版本控制 (p502)：在软件开发过程中，同一配置可能有多个版本。比如同一版本的错误的修正、改进、细化、扩展等会导致多个版本产生。提供一种有效的方法来区分和管理多个版本，以及这些版本之间的关系和演变 (git)

6.变更控制 (p502)：简单来说，就是对基线的变更有复杂的手续：1) .书面请求 2.) 评估变更申请 3.) 提取配置项 4.) 修改配置项 5) 质量保证

4.3 软件过程改进

1. 提高软件开发能力的意义获得更多的软件项目订单；开发更高质量的软件产品；取得较高的经济效益和社会效益

2. 软件开发的能力成熟度：开发组织希望通过软件能力成熟度度量找到自己的优势和差距，为提高自己的软件开发能力提供科学依据；客户希望通过软件组织的能力成熟度度量来科学地考察软件开发组织，寻找合适的合作伙伴来接受软件项目。

3. 能力成熟度模型(Capability Maturity Model, CMM)：

1) . 初始级：开发过程是无序的，有时甚至是混乱的。其成功往往归功于个人或少数人的努力

2.) 可重复级：已经建立了基本的项目管理，开发类似的项目可以重复以前的成功。

3.) 定义级：已经全面完美地定义了管理过程和工程过程。执行一个定义良好和文件化的过程已经成为组织甚至个人的一种有意识的行为

4) . 管理级：软件过程 and 产品质量可以定量地度量和控制。软件开发成本、进度和质量都是可以定量预测的。

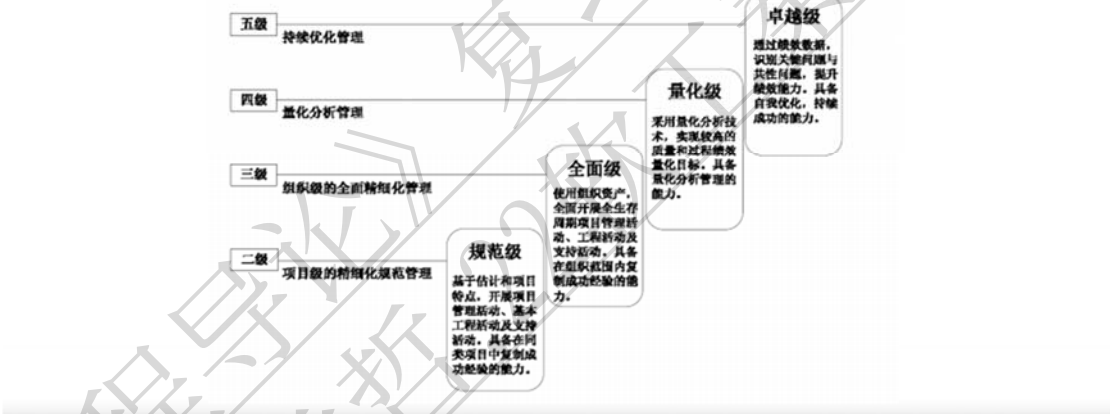
5) . 优化级：关注并可以利用反馈信息对过程 and 产品质量进行定量控制。能够采用先进的理念、方法和技术，持续改进软件过程，提高组织的软件开发和加工能力

3.4 关键过程域和关键实践

水平	关键过程域		
	管理	组织	工程
L1初始水平			
L2可重复等级	- 需求管理软件项目计划软件项目跟踪和监督软件分包管理软件质量保证软件配置管理		
L3定义级别	- 集成软件管理 - 团体之间的协调	- 组织过程重点组织过程定义培训计划	- 软件产品过程同行评审
L4管理级别	- 定量流程管理		- 软件质量管理
L5优化级别		- 技术更新管理流程变更管理	- 缺陷预防

国家标准：

❑ **Five Levels: Initial, Standard, Comprehensive, Quantitative, Outstanding**



4.4 软件维护和演化（P451 开始）

1.什么是软件维护？（p452）软件交付使用后，由于应用需求和环境的变化以及自身存在的问题而对软件系统进行改造和调整的过程

2.软件维护分类：纠正性，改善型，适应性，预防性（p452）

3.软件维护特点：（p455）同步性，周期长，费用高，难度大。

4.软件演化：软件是不断变化的，只有少数变化属于“软件维护”的范畴，而更多的变化属于“软件演化”的范畴。软件演化是指对软件进行大规模的功能增强和结构调整，以满足不断变化的软件需求或提高软件系统的质量，其粒度较大

5.软件演化特点（p457）

6.软件维护和演化技术（p460）：代码重组，逆向工程，设计重构，软件再工程。

总结：

1.软件项目管理

对软件项目中的产品、过程，人员进行测量、分析、计划、组织和控制，

2.软件配置管理

配置项的控制、记录、报告、验证、审核

3.软件过程改进

CMM：五个级别:初始、可重复、定义、管理、优化

4.软件维护和演化

纠正性、适应性、完整性、预防性维护

代码重组、逆向工程、设计重构、再工程

四、软件项目管理

— Software Project Management

(一) What is “Project”?何为项目

1 项目概念:

项目是指为**创建**一个**唯一**的**产品**或者提供唯一的服务而进行的**努力**

项目是基于既定**资源与约束**，为实现既定**目标**而实施的**活动**，它是一份临时工作，目的是创造独特产品、服务或者结果

典型项目示例: 阿波罗登月项目、Windows 7 开发项目、三峡水利项目、载人飞船项目，鸿蒙

项目也称工程，存在约束，实施活动，提供产品和服务

项目特点

目标性: 获得预期的结果

进度性: 在限定期间完成

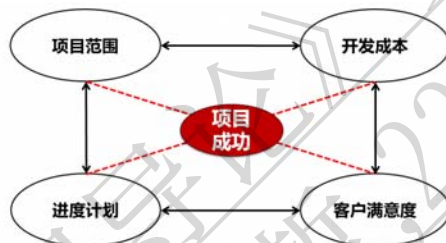
约束性: 具有有限的资源（如人员、经费、工具等）

多方性: 涉及多个不同人与组织

独立性: 项目间无重复性

不确定性: 项目的实施及其结果不确定性

2 影响项目成功的因素



项目的开展和实施受多要素影响，其结果具有不确定性

(二) 软件项目及其特点

软件项目: 针对软件这一特定产品和服务的项目努力开展“软件开发活动”

软件项目的特点

对象: 作为逻辑产品的软件

过程: 不以制造为主，没有重复生产过程

属性: 实施要素难以度量和估算，如成本、进度、质量

复杂性: 作为逻辑产品的复杂性非常高

易变性: 软件需求通常难以确定且经常变化

1 软件项目的任务

按照预定的进度、成本和质量，开发出满足用户要求的软件产品

用户需求、确保质量、成本限制、进度限制

目标性、进度性、约束性、多方性、独立性、不确定性

软件项目示例：火炮的火控软件、一体化指挥信息系统、卫星图像处理软件、导弹飞控软件、机载软件系统、微信、12306

2 军用软件项目的特点

□软件形态

✓往往与特定的物理或者硬件环境相互关联，**人机物融合系统**

□软件需求

✓环境的恶劣性和多变性，以及军用软件对**质量**提出更高要求，需要更高的**可靠性、安全性、灵活性**等

✓由于需要对事件作出及时响应，军用软件项目对系统的**实时性**提出更高的要求

□软件复杂性

✓随着信息化的开展，军用软件项目的**规模越来越大**，甚至可能是系统之系统，或者超大规模系统

3 软件项目设计的对象



4 软件项目管理

对软件项目所涉及的**过程、人员、产品、成本和进度**等要素进行**度量、分析、规划、组织和控制**的过程，以确保软件项目按照预定的成本、进度、质量要求顺利完成。

软件项目管理的对象：过程：怎么做(How)人员：谁来做(Who)产品：结果(What)

5 软件项目的管理要素

1.管理软件过程

明确软件开发活动及过程：**过程定义**

估算软件项目工作量成本：**软件度量**

制定计划、跟踪过程、风险控制等

2.管理软件产品

明确有哪些产品，呈什么形式(规范文档)

质量保证、配置管理、需求管理，风险控制

3.管理项目人员

组建开发团队、调动积极性和激情

团队建设、沟通、机制设计、风险控制

管理对象	人员	过程	产品
管理内容	团队建设和管理 纪律和激励机制	过程定义 软件度量 项目计划 项目跟踪	软件质量管理 软件配置管理 软件需求管理
	风险管理		

二 Software Configuration Management 软件配置管理

软件配置项（SCI）是软件生命周期内产生、需进行配置管理的工作产品

文档：需求规格说明书，设计规格说明书，项目计划，配置管理计划

代码：源代码、中间代码、可执行代码、...

数据：配置数据、数据库、数据文件、...

标准和规约：编码规范、...

在软件生命周期中对 **SCI** 进行的以下工作：系统地控制 **SCI** 的标识、存储、更动和发放；记录、报告其状态；验证 **SCI** 的正确性和一致性；对上述工作的审计

基线的概念

已经通过正式复审和批准的软件产品、标准或规约，它们可以作为进一步开发的基础；只能通过正式的变化控制过程才允许对它们进行变更

基线示例：经过评审后的，发现的问题已经得到纠正，用户和项目组双方认可并且正式批准，就可纳入基线

引入基线可以有效地控制对软件配置项（**SCI**）的更改，确保软件工件保持一定程度的稳定性。

基线中包含的配置项需要经过正式审核和批准。

不允许随意、非正式地更改基线，以确保基线相对稳定。

如果需要更改基线，则需要进行正式、严格的评估和批准。

版本控制

在软件开发过程中，同一个配置项可能会存在多个版本。

错误纠正、改进、完善、扩展等操作会导致产生多个版本。

提供一种有效的手段来区分和管理软件配置项的多个版本，以及这些版本之间的关系和演变。

例如 Git、SVN 等版本控制工具。

变更控制

软件配置管理必须控制对软件配置项（**SCIs**）的任何更改。

应对基线中的 **SCIs** 进行定期的变更程序。

提交书面变更请求 QA 评估变更请求 QA 提取 **SCIs** QA 修改 **SCIs** QA 质量保证 QA 添加到基线

软件过程改进

软件开发能力：

软件开发组织的能力直接影响到软件产品的质量、项目的进度和预算的执行；

软件开发组织需要提高团队、技术、环境、文化和管理水平：

提高软件开发能力；获得更多的软件项目订单；开发更高质量的软件产品；取得较高的经济效益和社会效益；

软件开发组织的能力成熟度度量是非常重要的。

软件功能的成熟度:

软件开发组织希望通过软件能力成熟度度量找到自身的优势和差距, 为提高软件开发能力提供科学依据;

软件客户希望通过软件组织的能力成熟度度量来科学地考察软件开发组织, 寻找合适的合作伙伴来接受软件项目

CMM 的产生和发展: CMM(能力成熟度模型)是由卡内基梅隆大学(CMU)的软件工程研究所(SEI)提出的;1986 年, MITRE 公司的 SEI 开始了 CMM 的工作, 为软件开发组织的能力提供评估, 帮助组织改进他们的软件过程;1987 年, 开发了两种方法(软件过程评估和软件成熟度评估)和成熟度问卷;1991 年, SEI 将成熟度框架演变为成熟度模型 1.0 版本。1993 年, CMM 的 1.1 版本发布, 并被 DoD 和 NASA 等政府部门广泛使用;1999 年, 美国国防部规定承担大型 DOD 软件项目的承包商必须拥有 CMM 成熟度 3 级认证

CMM 的等级: CMM 将软件能力成熟度分为五个级别

5.优化级别 4:管理级别 3:定义级别 2:可重复级别 1:初级级别

- 1: 开发过程是无组织的, 有时是混乱的; 成功往往归功于个人或少数人的努力
- 2: 是否建立了基本的项目管理; 开发类似的项目可以重复以往的成功
- 3: 已经完美地定义了它的管理过程和工程过程; 执行一个定义良好且文档化的过程已经成为组织, 甚至个人的一种有意识的行为
- 4: 软件过程和产品质量可以定量测量和控制; 软件开发成本、进度和质量是定量可预测的。
- 5: 重视并能利用反馈信息对过程 and 产品质量进行定量控制; 能够采用先进的思想、方法和技术, 持续改进软件过程, 提高组织的软件过程能力

关键域与关键实践: 如何评估软件开发组织的软件能力成熟度级别?

CMM 提供了 18 个关键过程域, 这些过程域定义了要达到的目标、要完成的任务以及要为每个程度级别进行的活动;

每个关键过程域都包含一组关键实践, 为了达到关键过程域的相应目标, 必须完成这些关键实践

GJB 5000B-2021

中华人民共和国军用软件能力成熟度模型国家标准

5 个等级:初级、标准、综合、定量、优秀

软件维护与发展

软件维护: 设备在使用过程中会出现故障, 需要进行维护。

软件交付使用后, 由于应用需求和环境的变化以及自身存在的问题, 对软件系统进行改造和调整的过程。

故障, 不能正常工作: 潜在的缺陷会产生软件错误; 需要纠正这些缺陷

服务变更, 需要升级: 软件需求已经改变; 需要加强软件和服务的功能

经营环境发生变化, 需要适应: 软件需要修改才能在新环境中运行

软件维护的形式

维修保养 适应性维护 完整性维护 预防性维护

什么是正确的维护? 纠正软件中的缺陷和错误

原因: 用户在使用软件过程中发现缺陷, 会向开发人员提出纠正性维护要求

目标：诊断和纠正软件系统的潜在缺陷

什么是适应性维护?修改软件以适应新的环境和平台

原因：软件运行在特定的环境中(硬件、操作系统、网络等)，运行环境会发生变化

目标：维护软件以适应环境变化

什么是完整性维护?软件的修改是为了增加新的功能和修改现有的功能

原因：在软件系统运行期间，用户可以要求增加新的功能，建议修改现有功能，或建议其他改进

目标：对软件系统进行改进和补充，以满足用户日益增长的需求

什么是预防性维修?修改软件结构，提高软件的可靠性和可维护性

原因：为了进一步提高软件系统的可维护性和可靠性，为今后软件改进的维护活动奠定基础

目标：获取软件结构，再改进软件结构

软件进化（演化）：交付的软件是不断变化的，只有少数变化属于“软件维护”的范畴，而更多的变化属于“软件进化”的范畴。软件进化是指为了满足不断变化的软件需求或提高软件系统的质量，对软件进行大规模的功能增强和结构调整：

功能增强的粒度很大 积极应对变化 连续性的发展 导致版本变更

概念	功能增强粒度	应对变化的方法	持久性或间隔性	版本变化
软件演化	粗粒	主动	持久性	是的
软件维护	细粒度	被动地	时间间隔	也许

软件维护演进技术：

1.代码重组;2.逆向工程;3.设计重构;4.再造工程

1: 在不改变软件功能的前提下，对程序代码进行重组，使重组后的代码具有更好的可维护性，并能有效地支持代码的变更

2: 在较低抽象层次的软件构件的基础上，通过对其理解和分析，生成较高抽象层次的软件构件;

通过对程序代码进行反向分析，生成与代码一致的设计模型和文档;

在了解程序代码和设计模型的基础上，分析软件系统的需求;

模型和文档典型应用场景:分析现有的程序，寻找比源代码更高层次的抽象(比如设计甚至需求)。

3: 缺少软件设计文档，说明软件文档与程序代码不一致，或者软件设计的内容不详细;软件维护工程师可以使用设计重构的方法来获取软件设计文档信息;

是逆向工程的一种特殊形式

4: 通过分析和更改软件体系结构来实现更高质量软件系统的过程;

再造包括逆向工程和正向工程